

CS-3510-B Problem Set 5

Elijah Martin McCauley

TOTAL POINTS

70 / 75

QUESTION 1

1 Problem 1 25 / 25

✓ - **0 pts** *Correct*

Table Definition (5 pts)

- **0 pts** Dimensions missing (no penalty)
- **1 pts** Minor mistake
- **3 pts** Mostly incorrect table definition.
- **5 pts** Table Definition Missing/Completely

incorrect

Base Cases and Recurrence (10 pts)

- **1 pts** Minor mistake in base case/ Missing one base case

- **2 pts** Base Case Missing/Completely Incorrect

- **1 pts** Minor mistake in recurrence

- **2 pts** Recurrence does not align with the table defined.

- **2 pts** Explanation of correctness Missing

- **6 pts** Recurrence Missing/Completely incorrect

- **4 pts** Major mistake in recurrence relation

- **10 pts** Non-DP Approach

Runtime Analysis (5 pts)

- **1 pts** Minor mistake

- **2 pts** Correct runtime with no/incorrect analysis.

- **3 pts** Correct runtime and analysis but suboptimal solution.

- **5 pts** Missing/Incorrect Runtime and analysis

Pseudocode (5 pts)

- **1 pts** Minor Error (typo, missing an index, etc)

- **2 pts** Code does not use the recurrence presented/ does not align with the table defined.

- **3 pts** Multiple minor errors or one major error that breaks correctness.

- **4 pts** Mostly incorrect/ dynamic programming isn't used

- **5 pts** Pseudo-code Missing

- **10 pts** Inefficient solution (higher than $\$O(n)\$$)

- **25 pts** Missing Answer

QUESTION 2

2 Problem 2 22 / 25

- **0 pts** Correct

Table Definition (5 pts)

- **0 pts** Dimensions missing (no penalty)

✓ - **3 pts** *Mostly incorrect table definition.*

- **5 pts** Table Definition Missing

- **1 pts** Minor mistake in definition.

Base Cases and Recurrence (10 pts)

- **1 pts** Minor mistake in base case/ Missing one base case

- **2 pts** Base Case Missing / incorrect in pseudocode

- **1 pts** Minor mistake in recurrence
- **2 pts** Recurrence does not align with the table defined.
- **2 pts** Explanation of correctness Missing
- **6 pts** Recurrence Missing
- **4 pts** Recurrence mostly incorrect

Pseudocode (5 pts)

- **1 pts** Minor Error (typo, missing an index, etc)
- **2 pts** Code does not use the recurrence presented/ does not align with the table defined / does not use correct base cases.
- **3 pts** Multiple minor errors or one major error that breaks correctness.
- **4 pts** Mostly incorrect/ dynamic programming isn't used
- **5 pts** Pseudo-code Missing
- **3 pts** There is an explanation but not pseudocode

Runtime Analysis (5 pts)

- **1 pts** Minor mistake
- **2 pts** Correct runtime with no/incorrect analysis.
- **3 pts** Correct runtime and analysis but suboptimal solution.
- **5 pts** Missing/Incorrect Runtime and analysis
- **25 pts** Missing Answer

1 $T[i, j]$ should not represent the lengths of the substring/subsequence (which are not necessarily explicitly known) but rather, the some notion of the problem restricted to the *prefixes* $X_1, \dots, X_i$ and $Y_1, \dots, Y_j$.

In addition, it must end in X_i .

QUESTION 3

3 Problem 4 23 / 25

- **0 pts** Correct

Table Definition (5 pts)

- **0 pts** Dimensions missing (no penalty)
- **1 pts** Minor mistake
- **3 pts** Mostly incorrect table definition.
- **5 pts** Table Definition Missing

Base Cases and Recurrence (10 pts)

- **1 pts** Minor mistake in base case/ Missing one base case
- **2 pts** Base Case Missing
- **1 pts** Minor mistake in recurrence
- **2 pts** Recurrence does not align with the table defined.
- **2 pts** Explanation of correctness Missing
- **6 pts** Recurrence Missing
- **4 pts** major mistakes in recurrence

Pseudocode (5 pts)

- **1 pts** Minor Error (typo, missing an index, etc)
- **2 pts** Code does not use the recurrence presented/ does not align with the table defined.
- **3 pts** Multiple minor errors or one major error that breaks correctness.
- **4 pts** Mostly incorrect/ dynamic programming isn't used
- **5 pts** Pseudo-code Missing
- **0 pts** Click here to replace this description.

Runtime Analysis (5 pts)

- **1 pts** Minor mistake

- **2 pts** Correct runtime with no/incorrect analysis.

- **3 pts** Correct runtime and analysis but suboptimal solution.

- **5 pts** Missing/Incorrect Runtime and analysis

- **25 pts** Missing Answer

- **2** *Point adjustment*

• errors in recurrence

Problem 1

(25 points)

Malav loves Insomnia cookies! Every day that he is allowed into the store, he goes in and buys all the boxes of cookies the store has in stock that day. Unfortunately, there are two problems:

1. Insomnia only has a finite number of cookie boxes available each day, and the cookie stock changes from day to day.
2. Insomnia has a policy that if you buy all the cookies from the store in one day, you can't come back the following day.

Fortunately, Malav knows how many cookie boxes will be in stock each day. The array $A = [a_1, \dots, a_k]$ of positive integers tells us that on day 1 there will be a_1 cookie boxes available, on day 2 there will be a_2 boxes, and so on.

Design a dynamic programming algorithm that maximizes the number of cookie boxes Malav can buy over the k days.

Example:

Input: $A = [5, 7, 8, 10, 9, 4, 2, 11, 6]$

Output: 33

Explanation: Malav can buy 5 boxes on the 1st day, 8 on the 3rd day, 9 on the 5th day and 11 boxes on the 8th day.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: The table would be $T[i]$ with length k where k is the number of days. The entries are $T[i] =$ maximum amount of cookies Malav could have bought by the i th day.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: $T[i] = \max(T[i-1], T[i-2] + A[i])$. This is correct because you cannot buy cookies two days in a row, so on a given day, the most cookies you can have is either the amount you had two days ago plus the amount you have today or the amount you have yesterday because if you buy cookies yesterday then you cannot buy cookies today.

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def maxcookies(A)
    allocate T[0, ..., k - 1]
    T[0] = A[0]
```

```
T[1] = max(A[1], T[0])
for i = 2 to k - 1:
    if T[i-1] > T[i-2] + A[i]:
        T[i] = T[i-1]
    else:
        T[i] = T[i-2] + A[i]
return T[k - 1]
```

(d) Analyze the running time of your algorithm.

Solution: The runtime of this algorithm is $\mathcal{O}(k)$ because we do a singular loop of length k where we are just comparing two values.

1 Problem 1 25 / 25

✓ - 0 pts Correct

Table Definition (5 pts)

- 0 pts Dimensions missing (no penalty)
- 1 pts Minor mistake
- 3 pts Mostly incorrect table definition.
- 5 pts Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one base case
- 2 pts Base Case Missing/Completely Incorrect
- 1 pts Minor mistake in recurrence
- 2 pts Recurrence does not align with the table defined.
- 2 pts Explanation of correctness Missing
- 6 pts Recurrence Missing/Completely incorrect
- 4 pts Major mistake in recurrence relation
- 10 pts Non-DP Approach

Runtime Analysis (5 pts)

- 1 pts Minor mistake
- 2 pts Correct runtime with no/incorrect analysis.
- 3 pts Correct runtime and analysis but suboptimal solution.
- 5 pts Missing/Incorrect Runtime and analysis

Pseudocode (5 pts)

- 1 pts Minor Error (typo, missing an index, etc)
- 2 pts Code does not use the recurrence presented/ does not align with the table defined.
- 3 pts Multiple minor errors or one major error that breaks correctness.
- 4 pts Mostly incorrect/ dynamic programming isn't used
- 5 pts Pseudo-code Missing
- 10 pts Inefficient solution (higher than $O(n)$)
- 25 pts Missing Answer

Problem 2

(25 points)

You are given two strings X and Y , of length n and m respectively. You want to find the length of the longest common “substring” of these two strings, i.e., you want to find the length of the longest string that appears as a substring of X and a subsequence of Y .

Example:

If $X = \text{“helloworld”}$ and $Y = \text{“longwayhome”}$, you should return 4, since “lowo” is the longest string that appears as a substring of X and as a subsequence of Y .

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: $T[i, j]$ where $i [0, \dots, m]$, $j [0, \dots, n]$ so X is the vertical and Y is the horizontal. The value in $T[i][j]$ is the current length of the substring of X that is in a subsequence of Y . By having i go all the way to m and j go to n , we can have the first column and row be all zeros, so we have a value to add 1 to if the first letter in X is in Y and vice versa.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: If we have a match of $X[j] = Y[i]$, $T[i, j] = T[i-1, j-1] + 1$. Else, $T[i][j] = T[i-1][j]$. Basically, if we find a match, we want to check if it is part of the current substring. If the value in $T[i-1][j-1]$ is greater than 0, then there was a match in the previous row and it was earlier in Y than the current match meaning we are in a substring of X that is a subsequence of Y .

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def substringsequence(X, Y)
    allocate T[0, ..., m][0, ..., n]
    maximumlength = 0
    for i = 0 to m
        T[i][0] = 0
    for j = 0 to n:
        T[0][j] = 0
    for j = 1 to n:
        for i = 1 to m:
            if X[j] == Y[i]:
                T[i][j] = T[i-1][j-1] + 1
            if T[i][j] > maximumlength:
                maximumlength = T[i][j]
```

```
        else:  
            T[i][j] = T[i-1][j]  
    return maximumlength
```

(d) Analyze the running time of your algorithm.

Solution: The runtime for this algorithm would be $\mathcal{O}(nm)$ where n is $X.length$ and $m = Y.length$. This is because we loop over all the letters in Y , n times in order to compare each letter.

2 Problem 2 22 / 25

- 0 pts Correct

Table Definition (5 pts)

- 0 pts Dimensions missing (no penalty)

✓ - 3 pts *Mostly incorrect table definition.*

- 5 pts Table Definition Missing

- 1 pts Minor mistake in definition.

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one base case

- 2 pts Base Case Missing / incorrect in pseudocode

- 1 pts Minor mistake in recurrence

- 2 pts Recurrence does not align with the table defined.

- 2 pts Explanation of correctness Missing

- 6 pts Recurrence Missing

- 4 pts Recurrence mostly incorrect

Pseudocode (5 pts)

- 1 pts Minor Error (typo, missing an index, etc)

- 2 pts Code does not use the recurrence presented/ does not align with the table defined / does not use correct base cases.

- 3 pts Multiple minor errors or one major error that breaks correctness.

- 4 pts Mostly incorrect/ dynamic programming isn't used

- 5 pts Pseudo-code Missing

- 3 pts There is an explanation but not pseudocode

Runtime Analysis (5 pts)

- 1 pts Minor mistake

- 2 pts Correct runtime with no/incorrect analysis.

- 3 pts Correct runtime and analysis but suboptimal solution.

- 5 pts Missing/Incorrect Runtime and analysis

- 25 pts Missing Answer

1 $T[i, j]$ should not represent the lengths of the substring/subsequence (which are not necessarily explicitly known) but rather, the some notion of the problem restricted to the *prefixes* $X_1, \dots, X_i$ and $Y_1, \dots, Y_j$.

In addition, it must end in $\$X_i\$$.

Problem 4

(25 points)

We call a sequence of integers $a_1, \dots, a_n$ *noisy* when the signs of the differences between two consecutive terms in the sequence strictly alternate between $+$ and $-$ (the difference is never zero). So the sequence either follows $a_1 < a_2 > a_3 < a_4 > \dots$ or it follows $a_1 > a_2 < a_3 > a_4 < \dots$. You are given an array of integers $A = [a_1, \dots, a_n]$.

Design a dynamic programming algorithm to find the length of the longest *noisy subsequence* in A .

Example:

Input: $A = [2, 4, -1, -5, -9, 7, 9, 0, 5, 5, -2]$

Output: 7

Explanation: An example of a *noisy subsequence* from the above array A is $2, 4, -1, 9, 0, 5, -2$. There are more examples, but the length is the same for all. On the other hand, $2, 4, 7, 9, 0, 5, 5$ is not a noisy subsequence because the differences between the three consecutive elements 2, 4, and 7 do not alternate. Two 5's also show up at the end of the sequence causing the consecutive difference to be zero.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: The table is $T[i][j]$ where i is $[0 \dots A.length - 1]$ and j is $[0, 1]$. $T[i][1]$ is the sign of the change between the previous entry of A and the current entry of A . $T[i][0]$ is the current length of the longest noisy subsequence.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: For each entry in the table we need to first find the sign of the change in entries. Then, if the sign is the same as the previous sign, $T[i][0] = T[i-1][0]$ because the noisy subsequence cannot be getting longer because to get longer it requires signs to be changing. If the signs are different, then the noisy subsequence is getting longer so $T[i][0] = T[i-1][0] + 1$.

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def noisysubsequence(A)
    allocate T[0, ..., A.length - 1][0, 1]
    T[0][0] = 1
    T[0][1] = 0
    for i = 1 to A.length - 1:
        if A[i] - A[i-1] > 0:
            T[i][1] = 1
```

```
    else if A[i] - A[i-1] < 0:
        T[i][-1] = -1
    else:
        T[i][1] = T[i-1][1]
    if T[i][1] == T[i-1][1]:
        T[i][0] = T[i-1][0]
    else:
        T[i][0] = T[i-1][0] + 1
return T[A.length - 1][0]
```

- (d) Analyze the running time of your algorithm.

Solution: The runtime of this algorithm would be $\mathcal{O}(A.length)$. This is because we run a singular loop over the length of A where we check the difference between two numbers and then update the T table.

3 Problem 4 23 / 25

- 0 pts Correct

Table Definition (5 pts)

- 0 pts Dimensions missing (no penalty)
- 1 pts Minor mistake
- 3 pts Mostly incorrect table definition.
- 5 pts Table Definition Missing

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one base case
- 2 pts Base Case Missing
- 1 pts Minor mistake in recurrence
- 2 pts Recurrence does not align with the table defined.
- 2 pts Explanation of correctness Missing
- 6 pts Recurrence Missing
- 4 pts major mistakes in recurrence

Pseudocode (5 pts)

- 1 pts Minor Error (typo, missing an index, etc)
- 2 pts Code does not use the recurrence presented/ does not align with the table defined.
- 3 pts Multiple minor errors or one major error that breaks correctness.
- 4 pts Mostly incorrect/ dynamic programming isn't used
- 5 pts Pseudo-code Missing
- 0 pts [Click here to replace this description.](#)

Runtime Analysis (5 pts)

- 1 pts Minor mistake
- 2 pts Correct runtime with no/incorrect analysis.
- 3 pts Correct runtime and analysis but suboptimal solution.
- 5 pts Missing/Incorrect Runtime and analysis
- 25 pts Missing Answer
- 2 *Point adjustment*
 - errors in recurrence

Problem Set 5: Dynamic Programming

Prof. Abraham Ladha

Due: 03/08/2023 11:59pm

- Please **TYPE** your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.
- Please write pseudo-code for this problem set, unless otherwise stated to write the algorithm in English.
- Fastest (and correct) solutions are worth more credit.

Problem 3

(25 points)

In the land of bartertopia, people both buy and sell goods in exchange for other goods. Each item i has an exchange rate j to each other item given by $r\{i, j\}$, meaning you can exchange $r\{i, j\}$ units of item j for one unit of item i . Instead of directly exchanging one item for another, you can also exchange goods using other goods: $r\{a, b\} \cdot \dots \cdot r\{z, a\}$. Often these trades will result in less than one because of the cost of doing an exchange, but sometimes because of inefficiencies, these series of exchanges will result in having more of the original good than you started with.

You are given the set of exchange rates $r\{i, j\}$. For two items m and n , design a dynamic programming algorithm to find the most advantageous sequence of exchanges for converting item m to item n .

Hint: You should represent the items and rates by a graph whose edge lengths are real numbers.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: The entries of the table $T[i][j]$ would be the maximum exchange rate from the start to each node i , in j moves. The exchange rates are given by $r\{i, j\}$ where i and j are nodes in a graph and the weight of the edge between them is the number of units of j for you can receive for a single unit of i . $T[0][j]$ are set to one and the rest of the entries are initialized to -infinity because we want the first path that we find to a node to always be greater than the initialized one.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: For all (i, j) in E , $\log(\text{dist}(j)) = \max(\log(\text{dist}(j)), \log(\text{dist}(i)) + \log(r\{i, j\}))$. Basically, we are trying to find the maximum distance to j , so we need to find the max of the current distance to j and the current distance to i times the edge between i and j , but we cannot just multiply them, so we need to add the logs of $\text{dist}(i)$ and $r\{i, j\}$ then compare to the log of $\text{dist}(j)$.

- (c) Write your algorithm **in words** to solve this problem.

Solution: First we need to make a graph out of the set of exchange rates where in $r\{i, j\}$, i and j are nodes in the graph with the weight of the edge is the number of units of j for a single unit of i . Once we have this graph, we can run a modified Bellman Ford on it where it checks the max of the current path and the new path, to get the maximum exchange rate from m to n where $T[0][j]$ are set to one and the rest of the entries are initialized to -infinity and for all (i, j) in E , $\log(\text{dist}(j)) = \max(\log(\text{dist}(j)), \log(\text{dist}(i)) + \log(r\{i, j\}))$ which we repeat $|V| - 1$ times. Additionally, since we are trying to find the max path instead of value which is what Bellman Ford returns, we need to keep track of parent nodes, so every time we update a node's maximum distance, we record

it's parent to be the node on the other end of the exchange rate. Then we can simply retrace this path starting from the end point to its parent to its parent's parent etc. Then we reverse that path so it will start from m and end at n instead of vice versa.

(d) Analyze the running time of your algorithm.

Solution: The runtime of the algorithm would be the same as the runtime for Bellman Ford which is $\mathcal{O}(|V||E|)$ because we are simply running Bellman Ford on the graph we make from the items and their exchange rates.